

```
[1]: import pandas as pd

[2]: registrations = pd.DataFrame({'reg_id':[1,2,3,4], 'name':['Andrew','Bobo','Claire','David']})
logins = pd.DataFrame({'log_id':[1,2,3,4], 'name':['Xavier','Andrew','Yolanda','Bobo']})

[3]: registrations

[3]:   reg_id  name
0        1  Andrew
1        2   Bobo
2        3  Claire
3        4   David

[4]: logins

[4]:   log_id  name
0        1  Xavier
1        2  Andrew
2        3  Yolanda
3        4   Bobo

[5]: pd.merge(left = registrations, right = logins, how = 'outer', on = 'name')

[5]:   reg_id  name  log_id
0        1  Andrew    2.0
1        2   Bobo    4.0
2        3  Claire    NaN
3        4   David    NaN
4      NaN  Xavier    1.0
5      NaN  Yolanda    3.0
```

```
[6]: registrations = registrations.set_index('name')

[7]: registrations

[7]:   reg_id
name
Andrew    1
Bobo      2
Claire    3
David     4

[8]: logins

[8]:   log_id  name
0        1  Xavier
1        2  Andrew
2        3  Yolanda
3        4   Bobo

[9]: pd.merge(left = registrations, right = logins, how = 'outer')
```

```
-----
MergeError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 pd.merge(left = registrations, right = logins, how = 'outer')

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\reshape\merge.py:385, in merge(left, right, how, on, left_on, r
ight_on, left_index, right_index, sort, suffixes, copy, indicator, validate)
    371     _cross_merge(
    372         left_df,
    373         right_df,
    (...)  382         validate=validate,
    383     )
    384 else:
-> 385     op = _MergeOperation(
    386         left_df,
    387         right_df,
    388         how=how,
    389         on=on,
    390         left_on=left_on,
    391         right_on=right_on,
    392         left_index=left_index,
    393         right_index=right_index,
    394         sort=sort,
    395         suffixes=suffixes,
    396         indicator=indicator,
    397         validate=validate,
    398     )
    399     return op.get_result()

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\reshape\merge.py:1010, in _MergeOperation.__init__(self, left,
right, how, on, left_on, right_on, left_index, right_index, sort, suffixes, indicator, validate)
    1003     msg = (
    1004         "Not allowed to merge between different levels. "
    1005         f"({_left.columns.nlevels} levels on the left, "
    1006         f"({_right.columns.nlevels} levels on the right)"
    1007     )
    1008     raise MergeError(msg)
-> 1010 self.left_on, self.right_on = self._validate_left_right_on(left_on, right_on)
    1012 (
    1013     self.left_join_keys,
    1014     self.right_join_keys,
    (...)  1017     right_drop,
    1018 ) = self._get_merge_keys()
    1020 self.left_drop:

File c:\Users\pabal\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\reshape\merge.py:1899, in _MergeOperation._validate_left_right_
on(self, left_on, right_on)
    1897 common_cols = left_cols.intersection(right_cols)
    1898 if len(common_cols) == 0:
-> 1899     raise MergeError(
    1900         "No common columns to perform merge on. "
    1901         f"Merge options: left_on={left_on}, "
    1902         f"right_on={right_on}, "
    1903         f"left_index={self.left_index}, "
    1904         f"right_index={self.right_index}"
    1905     )
    1906 if (
    1907     not left_cols.join(common_cols, how="inner").is_unique
    1908     or not right_cols.join(common_cols, how="inner").is_unique
    1909 ):
    1910     raise MergeError(f"Data columns not unique: {common_cols}")

MergeError: No common columns to perform merge on. Merge options: left_on=None, right_on=None, left_index=False, right_index=False
```

Now, if we want to merge those together, we should not follow the above method, instead follow:

```
[11]: pd.merge(left = registrations, right = logins, left_index = True, right_on = 'name', how = 'inner')

[11]:   reg_id  log_id  name
1        1        2  Andrew
3        2        4   Bobo
```

This code merges the `registrations` and `logins` DataFrames by matching the `index` of `registrations` with the `name` column of `logins`. Since `how='inner'` is used, only the rows with matching names in both DataFrames are included in the result.

Let's see what happens if you're dealing with different key column names in the joined tables:

```
[12]: registrations = registrations.reset_index()

[13]: registrations

[13]:   name  reg_id
0  Andrew      1
1   Bobo      2
2  Claire      3
3  David      4

[14]: registrations.columns = ['reg_name','reg_id']

[15]: registrations

[15]:   reg_name  reg_id
0  Andrew      1
1   Bobo      2
2  Claire      3
3  David      4

[16]: logins

[16]:   log_id  name
0        1  Xavier
1        2  Andrew
2        3  Yolanda
3        4   Bobo

[18]: pd.merge(left = registrations, right = logins, how = 'inner', left_on='reg_name', right_on='name')

[18]:   reg_name  reg_id  log_id  name
0  Andrew      1        2  Andrew
1   Bobo      2        4   Bobo
```

We can see that it returned both `reg_name` and `name` even though they should be technically representing the same values.

So, let's try removing the duplicate.

```
[19]: pd.merge(left = registrations, right = logins, how = 'inner', left_on='reg_name', right_on='name').drop('reg_name',axis=1)

[19]:   reg_id  log_id  name
0        1        2  Andrew
1        2        4   Bobo
```

OR

```
[20]: pd.merge(left = registrations, right = logins, how = 'inner', left_on='reg_name', right_on='name').drop('name',axis=1)

[20]:   reg_name  reg_id  log_id
0  Andrew      1        2
1   Bobo      2        4

[21]: registrations.columns = ['name','id']

[22]: logins.columns = ['id','name']

[23]: registrations

[23]:   name  id
0  Andrew  1
1   Bobo  2
2  Claire  3
3  David  4

[24]: logins

[24]:   id  name
0    1  Xavier
1    2  Andrew
2    3  Yolanda
3    4   Bobo

[25]: pd.merge(left = registrations, right = logins, how = 'inner', on = 'name')

[25]:   name  id_x  id_y
0  Andrew    1    2
1   Bobo    2    4

id_x is for left table and id_y is for right table.

We can give own suffixes:

[27]: pd.merge(left = registrations, right = logins, how = 'inner', on = 'name',suffixes=(' registration',' login'))

[27]:   name  id registration  id login
0  Andrew    1      1      2
1   Bobo    2      2      4
```